Voronoi-clipped fixed-base scalar table

Contribution. An opt-in secp256k1 radix-256 evaluator intersects the terminal norm disk with three Eisenstein Voronoi support strips. Its terminal table has **11,023** unit-orbit entries versus **13,314** for the direct disk; the complete table has **174,883** stored point entries, saving **2,291 entries / 146,624 point-payload bytes**. The native audit records **11,505,936 retained bytes**.

Instance. secp256k1 over Fp, y^2=x^3+7, standard prime-order generator G. The exact p, subgroup order r, G, lattice basis U,V, and scalar fixtures are in REPORT-2026-10-12.md, search.json, and holdout.json. The domain is reduced public scalars 0 <= k < r. Up to sixteen point selections are used.

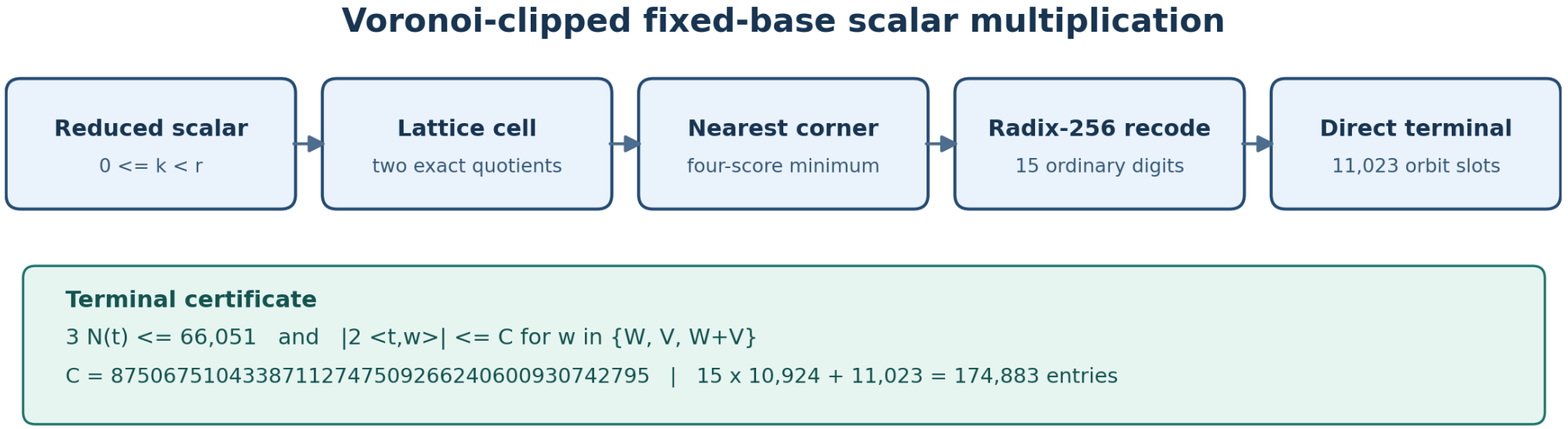
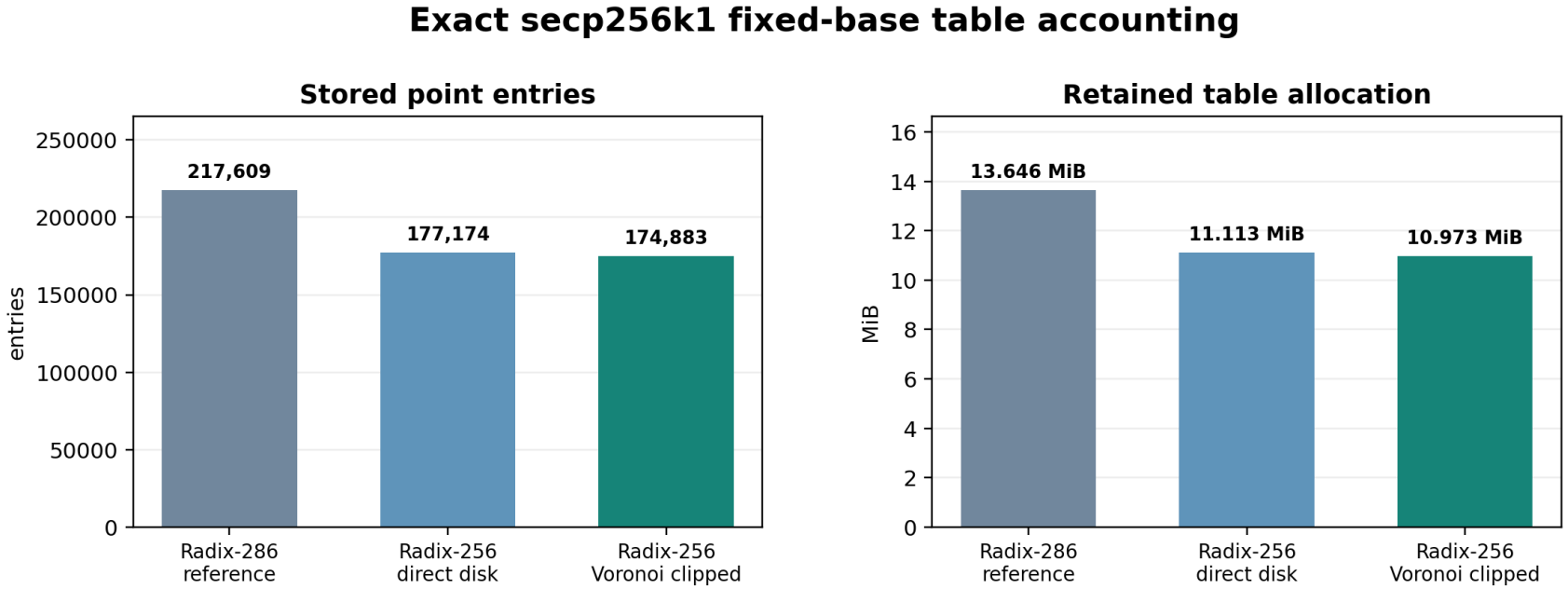


Figure 1. The exact terminal certificate combines 3N(t) <= 66,051 with three support strips. The editable source is figures/make_figures.py; figures/sources.json binds the source receipts.

Method. W=U-2V and W,V,W+V each have norm r. The four-corner minimum obeys the Voronoi inequalities; Cauchy bounds each ordinary radix-256 digit. Fifteen-step reconstruction bounds the terminal in all three strip directions. The Lean proof map and exact hypotheses are in PROOF.md.

Exact census and native evidence

Quantity	Direct disk	Voronoi clipped	Evidence
Terminal orbit entries	13,314	11,023	search.json; search.py
Total point entries	177,174	174,883	search.json; native audit
Retained table bytes	11,652,560	11,505,936	prior replay; storage-macos.json
Independent point cases	-	2,048 / 2,048	replay.json; macOS + Linux VM



One audited build per format; exact table counts and allocator accounting, sampling uncertainty not applicable.

Figure 2. Exact point counts and retained table accounting from one build per format. Sampling uncertainty does not apply to these integer counts. Native process RSS and placement are separate manifest fields.

Proof status, execution boundary, and sources

Checked proof and open bridge

Lean 4.34.1 checks the lattice-neighbor norms, exact four-corner score identity, minimum-corner Voronoi strips, a reciprocal/product two-unit coordinate enclosure, fraction-score intervals and a composed theorem taking a strict classifier winner into all six Voronoi strips and actual lattice-neighbor inequalities. Exact floor quotients and all four corner offsets satisfy the checked scaled-coordinate identity; Euclidean remainders establish normalized cell bounds. Lean also checks Cauchy digit support, fifteen-step reconstruction, and conditional terminal inclusion. lean-build.txt retains the build and axiom audit output. Lean checks both native reciprocal constants against the floor inequalities and the reduced-scalar range; check.py matches all fourteen words to the Rust source. Lean also checks exact-product extraction, bounded carry propagation, and an indexed outer pass over the native offset and reciprocal-sized array slice. Four such passes have zero final carry and compute the exact scalar-times-reciprocal product under bounded-word inputs. The checked modeled loop includes all intermediate prefix-product equalities and the eleven-word capacity proof. Formal correspondence to the native Rust loop remains open. One checked theorem composes the four-pass product with its modeled high three words, word seven, and low seven words as the exact quotient, fractional word, and remainder; the native return-index correspondence remains open. The fallback remains to be composed with the four-corner theorem; the finite 11,023-orbit census also lacks a Lean cardinality theorem. The exact exit criteria are in problems/prime-j0-hex-terminal-formal-coverage.md.

Native replay and placement

An exact-source wrapper compiled the native reciprocal word loop and matched all 5,197 deterministic cases against an independent Python integer product, including carry-heavy patterns and both production reciprocals. native-reciprocal-loop.json retains source, compiler, input, and output hashes. The source-frozen post-commit holdout has 512 distinct scalars disjoint from prior panels. Four 512-case panels match independently computed points on macOS ARM64; the same semantic receipt passed after offline Linux ARM64 Docker VM rebuild. storage-macos.json and storage-linux.json record CPU model/topology, cache exposure, affinity, cgroup limits, memory policy, co-runners, process RSS samples, and whether page placement was observable. The retained-byte value is internal exact table accounting, not resident memory.

Inconclusive measurements and next step

The available RunPod service is serial but its Docker/cgroup-v1 host has no isolated CPU partition or nohz_full set and currently has an active CryptoMiniSat job. The incidental audit wall and CPU times do not support an online speedup. The next proof step connects the exact 512-bit limb product to the checked two-unit interval theorem, then handles the fallback branch.

Canonical graphs and reproduction

Checked prime-j0-cost-aware-chain/orbit-graph-inputs.json, ec-arithmetic-optimizations/figures, and volcano-ic/figures. They encode graph, operation-layer, or isogeny facts unaffected by this scalar table. Rebuild with search.py --check, screen.py --check, check.py --check, and figures/make_figures.py. REPORT-2026-10-12.md links each exact receipt and source.

Source files: search.json; screen.json; replay.json; storage-macos.json; storage-linux.json; VoronoiTerminal.lean; PROOF.md; figures/sources.json.